Operating System Assignment 3

Name: Teesha Madan

Class: AIDS_F

Roll no: 68

Task 1: Create a Parent Process and Child Process

Aim

To create a parent process and a child process using the fork() system call and display their Process IDs (PID) and Parent Process ID (PPID).

Theory

The fork() system call creates a new child process by duplicating the parent process. After the fork, both parent and child execute independently. The parent receives the child's PID, while the child receives 0 as the return value of fork(). The functions getpid() and getppid() are used to obtain the Process ID and Parent Process ID respectively.

Program

```
 GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID  : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else
    {
        printf("Parent Process\n");
        printf("PID  : %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }

    return 0;
}
```

Output

```
admin@DESKTOP-6IGL1IF MINGW64 ~ $ nano q1.c
admin@DESKTOP-6IGL1IF MINGW64 ~ $ gcc q1. c -o q1
admin@DESKTOP-6IGL1IF MINGW64 ~ $ ./q1
Parent Process
PID  : 2266
Child PID : 2267
Child Process
PID  : 2267
PPID : 365
```

Task 2: Create 10 Child Processes

Aim

To create ten child processes using the fork() system call.

Theory

Multiple child processes can be created by repeatedly calling the fork() system call. Each child process receives a unique Process ID (PID) while sharing the same parent process. This demonstrates process creation in Linux.

Program

```
  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    for(int i=0;i<10;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID = %d Parent = %d\n",
                   i+1,getpid(),getppid());
            return 0;
        }
    }

    sleep(5);

    return 0;
}
```

Output

```
admin@DESKTOP-6IGL1IF MINGW64 ~ $ nano q2.c
admin@DESKTOP-6IGL1IF MINGW64 ~ $ gcc q2.c -o q2
admin@DESKTOP-6IGL1IF MINGW64 ~ $ ./q2
 Child 2 PID = 13260 Parent = 13258
 Child 1 PID = 13259 Parent = 13258
 Child 3 PID = 13261 Parent = 13258
 Child 4 PID = 13262 Parent = 13258
 Child 5 PID = 13263 Parent = 13258
 Child 6 PID = 13264 Parent = 13258
 Child 7 PID = 13265 Parent = 13258
 Child 8 PID = 13266 Parent = 13258
 Child 9 PID = 13267 Parent = 13258
 Child 10 PID = 13268 Parent = 13258
```

Task 3: Create Orphan Processes and Identify the New Parent

Aim

To create orphan processes by terminating the parent process and identify the new parent of the orphan processes.

Theory

An orphan process is created when its parent process terminates before the child process. The operating system automatically assigns a new parent to the orphan process. On a normal Linux system, the orphan is adopted by init/systemd (PID 1). In WSL, the orphan is adopted by the WSL /init process.

Program

```
  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    for(int i=0;i<5;i++)
    {
        if(fork()==0)
        {
            sleep(5);

            printf("Child PID = %d\n",getpid());
            printf("New Parent = %d\n",getppid());

            return 0;
        }
    }

    printf("Parent exiting...\n");
    exit(0);
}
```

Output

Task 4: Find the Priority of Running Processes

Aim

To determine the priority and scheduling information of running processes.

Theory

Linux assigns a priority to every process for CPU scheduling. The ps -el command displays process information such as Process ID (PID), Parent Process ID (PPID), Process State (S), Priority (PRI), and Nice Value (NI). These values are used by the scheduler to manage process execution.

Program

Command:

ps -el

Output

```
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0       1       0  0  80   0 -  6137 -      ?        00:00:01 systemd
5 S     0       2       1  0  80   0 -   793 -      hvc0     00:00:00 init-systemd(Ub
0 S     0       6       2  0  80   0 -   793 -      hvc0     00:00:00 init
4 S     0      43       1  0  79  -1 - 12595 -      ?        00:00:00 systemd-journal
4 S   989      80       1  0  80   0 -  5631 -      ?        00:00:00 systemd-resolve
4 S     0      87       1  0  80   0 -  8814 -      ?        00:00:01 systemd-udevd
4 S     0     106       1  0  80   0 -   720 -      ?        00:00:00 chronyd-starter
4 S     0     107       1  0  80   0 -  1116 -      ?        00:00:00 cron
4 S   995     108       1  0  80   0 -  2265 -      ?        00:00:00 dbus-daemon
4 S     0     129       1  0  80   0 - 11019 -      ?        00:00:00 networkd-dispat
4 S     0     159       1  0  80   0 -  4651 -      ?        00:00:00 systemd-logind
4 S     0     211       1  0  80   0 -  1371 -      tty1     00:00:00 agetty
4 S   988     227     106  0  80   0 -  5101 -      ?        00:00:00 chronyd
4 S     0     229       1  0  80   0 - 30860 -      ?        00:00:00 unattended-upgr
1 S   988     249     227  0  80   0 -  3019 -      ?        00:00:00 chronyd
5 S     0     364       2  0  80   0 -   797 -      ?        00:00:00 SessionLeader
5 S     0     365     364  0  80   0 -   797 -      ?        00:00:00 Relay(366)
4 S  1000     366     365  0  80   0 -  1565 do_wai pts/0    00:00:00 bash
4 S     0     367       2  0  80   0 -  2158 -      ?        00:00:00 login
4 S  1000     408       1  0  80   0 -  5613 -      ?        00:00:00 systemd
5 S  1000     414     408  0  80   0 -  5725 -      ?        00:00:00 (sd-pam)
4 S  1000     439     367  0  80   0 -  1532 do_sel pts/1    00:00:00 bash
4 S   987     676       1  0  80   0 - 76609 -      ?        00:00:00 polkitd
0 S  1000    2500     408  0  80   0 -  2132 do_epo ?        00:00:00 dbus-daemon
4 S   100    5257       1  0  80   0 - 55133 -      ?        00:00:00 rsyslogd
0 R  1000   13449     366  0  80   0 -  1796 -      pts/0    00:00:00 ps
```

Task 5: Create and Identify a Zombie Process

Aim

To create a zombie process and identify it using Linux commands.

Theory

A zombie process is a process that has completed execution but still has an entry in the process table because its parent process has not called the wait() system call to collect its exit status. Zombie processes are identified by the process state Z.

Program

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent sleeping for 20 seconds...\n");
        sleep(20);

        printf("Parent exiting...\n");
    }

    return 0;
}
```

Output

```
$ ./zombie
```

```
Parent sleeping for 20 seconds...
Child exiting...
Parent exiting...
```

```
$ ps -el | grep Z
```

```
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
1 Z  1000   13679   13678  0  80   0 -     0 -      pts/0    00:00:00 zombie
```

Task 6(a): Demonstrate the wait() System Call

Aim

To demonstrate the working of the wait() system call.

Theory

The wait() system call suspends the execution of the parent process until one of its child processes terminates. It is used for process synchronization and prevents the creation of zombie processes by collecting the child's exit status.

Program

```
  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    if(fork()==0)
    {
        printf("Child\n");
        return 0;
    }

    wait(NULL);

    printf("Parent resumes\n");
}
```

Output

```
admin@DESKTOP-6IGL1IF MINGW64 ~ /mnt/c/Users/vira_/OneDrive/Desktop/college/SEM5/OS$ nano q6.c
admin@DESKTOP-6IGL1IF MINGW64 ~ /mnt/c/Users/vira_/OneDrive/Desktop/college/SEM5/OS$ gcc q6.c -o q6
admin@DESKTOP-6IGL1IF MINGW64 ~ /mnt/c/Users/vira_/OneDrive/Desktop/college/SEM5/OS$ ./q6
Child
Parent resumes
```

Task 6(b): Demonstrate the execv() System Call

Aim

To demonstrate the working of the execv() system call.

Theory

The execv() system call replaces the currently executing process with a new program. It does not create a new process; instead, it loads and executes another program within the same process while retaining the same Process ID (PID).

Program

```
  GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    char *args[]={"ls","-l",NULL};

    execv("/bin/ls",args);

    return 0;
}
```

Output

```
admin@DESKTOP-6IGL1IF MINGW64 ~ $ nano q6_2.c
admin@DESKTOP-6IGL1IF MINGW64 ~ $ gcc q6_2.c -o q6_2
admin@DESKTOP-6IGL1IF MINGW64 ~ $ ./q6_2
total 144
-rwxrwxrwx 1 admin admin   12025 Jul  8 07:11 'Basic Commands.pdf'
-rwxrwxrwx 1 admin admin   45455 Jul  8 09:11 'Execute Basic Linux commands.pptx'
-rwxrwxrwx 1 admin admin  208629 Jul 29 10:45 'OS Syllabus_2026-27.docx'
-rwxrwxrwx 1 admin admin  251946 Aug  4 14:12 'Operating System Assignment 1.docx'
-rwxrwxrwx 1 admin admin  527731 Aug  4 14:17 'Operating System Assignment 2.docx'
-rwxrwxrwx 1 admin admin   11227 Jul 22 07:37 orphan.c
-rwxrwxrwx 1 admin admin   16120 Aug  5 11:10 q1
-rwxrwxrwx 1 admin admin     381 Aug  5 11:12 q1.c
-rwxrwxrwx 1 admin admin   16120 Aug  5 11:17 q2
-rwxrwxrwx 1 admin admin     285 Aug  5 11:17 q2.c
-rwxrwxrwx 1 admin admin   16200 Aug  5 11:19 q3
-rwxrwxrwx 1 admin admin     349 Aug  5 11:19 q3.c
-rwxrwxrwx 1 admin admin   16032 Aug  5 11:32 q6
-rwxrwxrwx 1 admin admin     203 Aug  5 11:31 q6.c
-rwxrwxrwx 1 admin admin   16000 Aug  5 11:33 q6_2
-rwxrwxrwx 1 admin admin     133 Aug  5 11:33 q6_2.c
-rwxrwxrwx 1 admin admin   16080 Aug  5 11:25 zombie
-rwxrwxrwx 1 admin admin     334 Aug  5 11:24 zombie.c
admin@DESKTOP-6IGL1IF MINGW64 ~ $
```